

Time Series Analysis

Havana Rika



Ben-Gurion University
of the Negev

Deep Learning Models for Time-Series

- Fully connected layers
- CNN
- RNN and LSTM
- Encoder - Decoder
- Attention and Transformers

Understanding the encoder-decoder paradigm

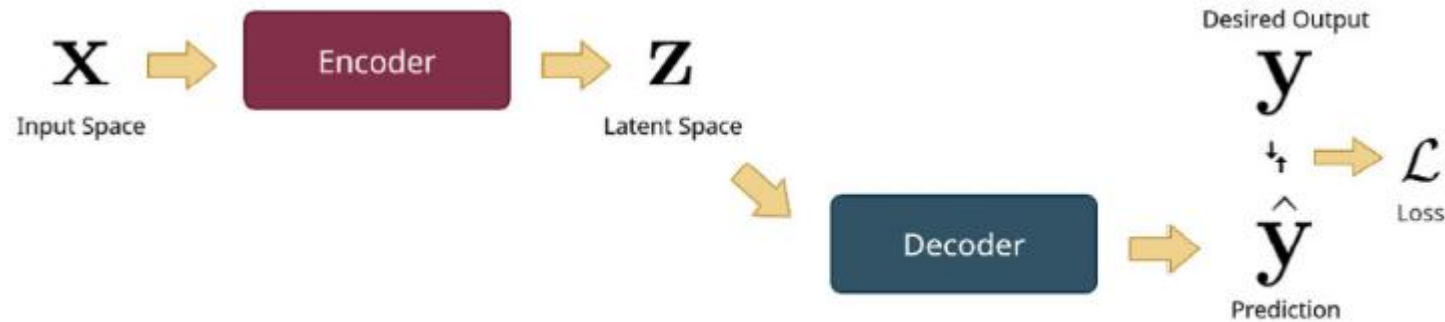


Figure 12.1: The encoder-decoder architecture

In the context of time series forecasting, the encoder consumes the history and retains the information that is required for the decoder to generate the forecast

$$y_t = h(y_{t-1}, y_{t-2}, \dots, y_{t-N})$$

Now, using the encoder-decoder paradigm, we can rewrite it as follows:

$$z_t = h(y_{t-1}, y_{t-2}, \dots, y_{t-N})$$

$$y_t = g(z_t)$$

Here, h is the encoder and g is the decoder.

Feed-forward networks

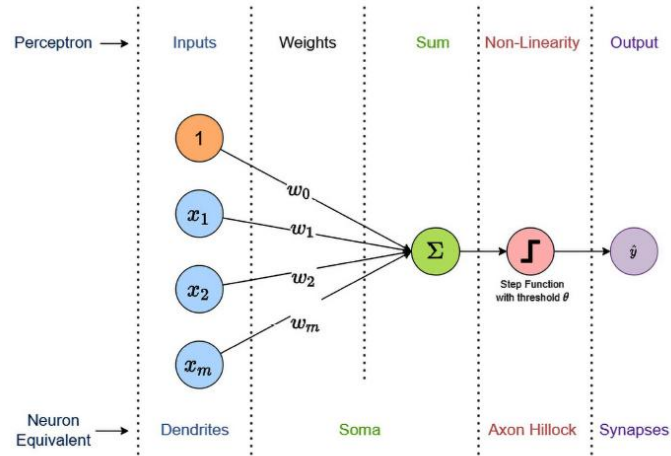


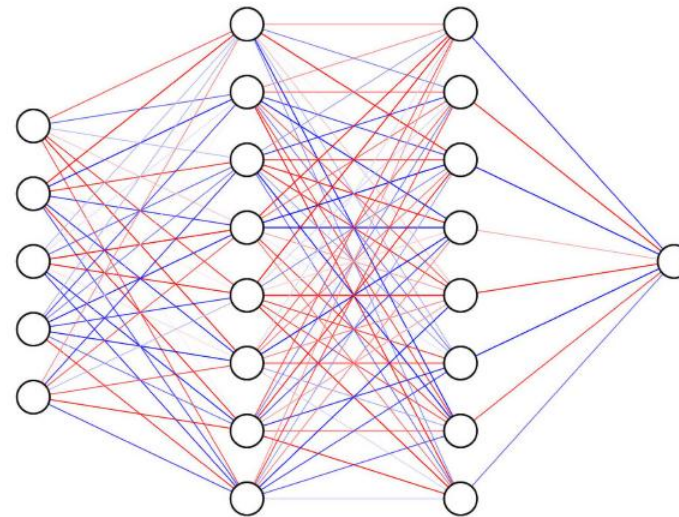
Figure 11.4: Perceptron

$$\hat{y} = g \left(w_0 + \sum_{i=1}^m x_i w_i \right)$$

Labels for the equation: Output ($\hat{y}$), Bias (w_0), Linear Combination of Inputs ($\sum_{i=1}^m x_i w_i$), Non-Linearity (g).

Activation functions: Relu, tanh, sigmoid
Output activation function: Softmax

Loss function
Forward and backward propagation
Gradient descent
Learning rate



Input Layer $\in \mathbb{R}^5$ Hidden Layer $\in \mathbb{R}^8$ Hidden Layer $\in \mathbb{R}^8$ Output Layer $\in \mathbb{R}^1$

Figure 12.2: A Feed Forward Network (FFN)

Feed-forward networks

We embed time and convert a time series problem into a regression problem before feeding it into the FFN. As a decoder, we use it on the latent vector (the output from the encoder) to get to the output

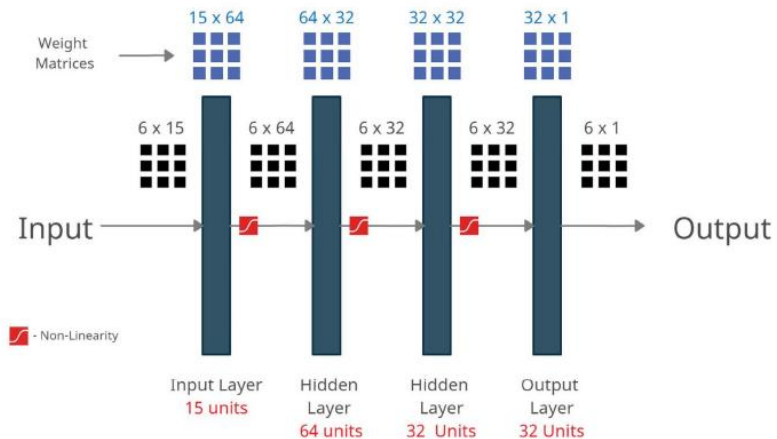
In the time series forecasting context, an FFN can be used as an encoder as well as a decoder.

As an encoder, we can use an FFN just like we used machine learning models

Feed-forward networks

```
N = 20
df = pd.DataFrame({
    "date": pd.date_range(periods=N, start="2021-04-12", freq="D"),
    "ts": np.random.randn(N)
})
```

```
ts = torch.from_numpy(df.ts.values).float()
window = 15
# Creating windows of 15 over the dataset
ts_dataset = ts.unfold(0, size=window, step=1)
```



```
# The FFN we define would have this architecture
# window(windowed input) >> 64 (hidden layer 1) >> 32 (hidden layer 2) >> 32
# (hidden layer 2) >> 1 (output)
ffn = nn.Sequential(
    nn.Linear(in_features=window,out_features=64), # (batch-size x window) -->
    (batch-size x 64)
    nn.ReLU(),
    nn.Linear(in_features=64,out_features=32), # (batch-size x 64) --> (batch-
    size x 32)
    nn.ReLU(),
    nn.Linear(in_features=32,out_features=32), # (batch-size x 32) --> (batch-
    size x 32)
    nn.ReLU(),
    nn.Linear(in_features=32,out_features=1), # (batch-size x 32) --> (batch-
    size x 1)
)
```

```
ffn(ts_dataset)
# or more explicitly
ffn.forward(ts_dataset)
```

Python practice - Fully connected network

```
pip install -U tensorflow
```

```
import pandas as pd
passengers = pd.read_csv(
    "passengers.csv", parse_dates=["date"]
).set_index("date")
```

```
import tensorflow as tf
import tensorflow.keras as keras
from tensorflow.keras.layers import Dense, Input, Dropout

DROPOUT_RATIO = 0.2
HIDDEN_NEURONS = 10

callback = tf.keras.callbacks.EarlyStopping(
    monitor='loss', patience=3
)
```

Python practice - Fully connected network

```
def create_model(passengers):
    input_layer = Input(len(passengers.columns))
    hidden_layer = Dropout(DROPOUT_RATIO)(input_layer)
    hidden_layer = Dense(HIDDEN_NEURONS, activation='relu')(hidden_layer)
    output_layer = Dropout(DROPOUT_RATIO)(hidden_layer)
    output_layer = Dense(1)(output_layer)
    model = keras.models.Model(
        inputs=input_layer, outputs=output_layer
    )
    model.compile(
        loss='mse',
        optimizer=keras.optimizers.Adagrad(),
        metrics=[keras.metrics.RootMeanSquaredError(), keras.metrics.
MeanAbsoluteError()])
    return model
```

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    passengers, passengers.passengers.shift(-1), shuffle=False
)
```

```
model = create_model(X_train)
model.fit(X_train, y_train, epochs=1000, callbacks=[callback])
```

We should get an output of the loss and the metrics at each epoch as they finish:

```
Epoch 1/1000
4/4 [=====] - 0s 3ms/step - loss: 71752.7812 - root_m
ean_squared_error: 267.8671 - mean_absolute_error: 237.6112
Epoch 2/1000
4/4 [=====] - 0s 2ms/step - loss: 69809.4141 - root_m
ean_squared_error: 264.2147 - mean_absolute_error: 243.9879
Epoch 3/1000
4/4 [=====] - 0s 2ms/step - loss: 76103.9375 - root_m
ean_squared_error: 275.8694 - mean_absolute_error: 253.2610
Epoch 4/1000
4/4 [=====] - 0s 2ms/step - loss: 76133.8672 - root_m
ean_squared_error: 275.9237 - mean_absolute_error: 253.2717
Epoch 5/1000
```


Python practice - Fully connected network

```
import matplotlib.pyplot as plt

def show_result(y_test, predicted):
    plt.figure(figsize=(16, 6))
    plt.plot(y_test.index, predicted, 'o-', label="predicted")
    plt.plot(y_test.index, y_test, '.-', label="actual")
    plt.ylabel("Passengers")
    plt.legend()
```

```
show_result(y_test, predicted)
```

Here's the graph:

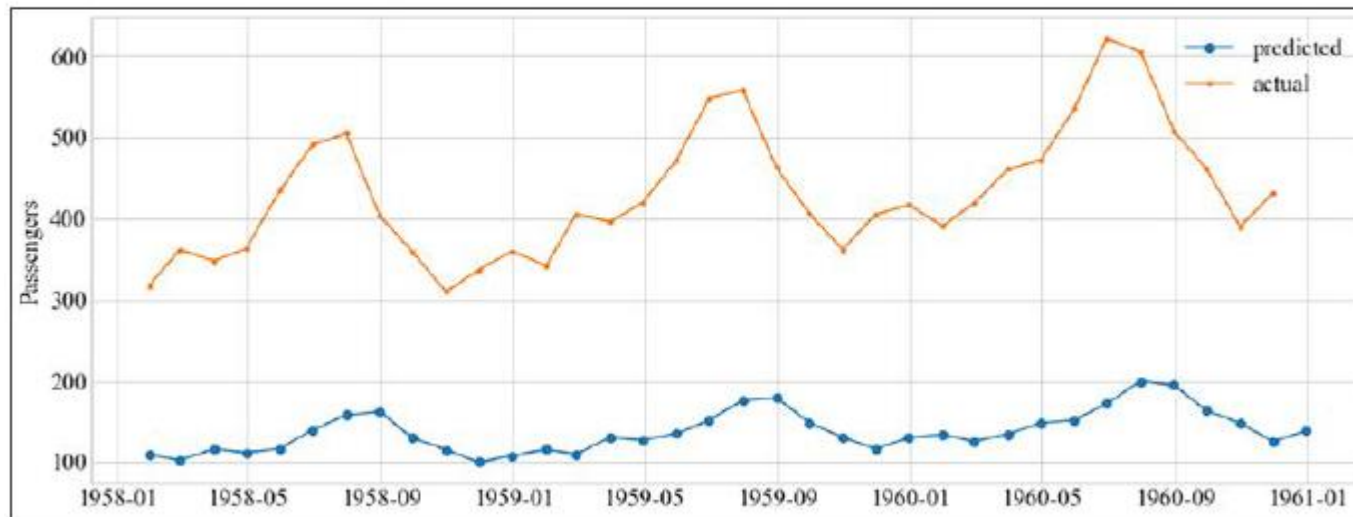


Figure 10.13: Predicted against actual airplane passengers: Naïve fully connected network

Include the year and the month as predictor variables.

```
passengers["month"] = passengers.index.month.values  
passengers["year"] = passengers.index.year.values
```

```
model = create_model(passengers)
```

And we are ready for another training round:

```
X_train, X_test, y_train, y_test = train_test_split(  
    passengers, passengers.passengers.shift(-1), shuffle=False
```

```
)  
model.fit(X_train, y_train, epochs=100, callbacks=[callback])  
predicted = model.predict(X_test)  
show_result(y_test, predicted)
```

Let's see how well the model predictions match the test data:

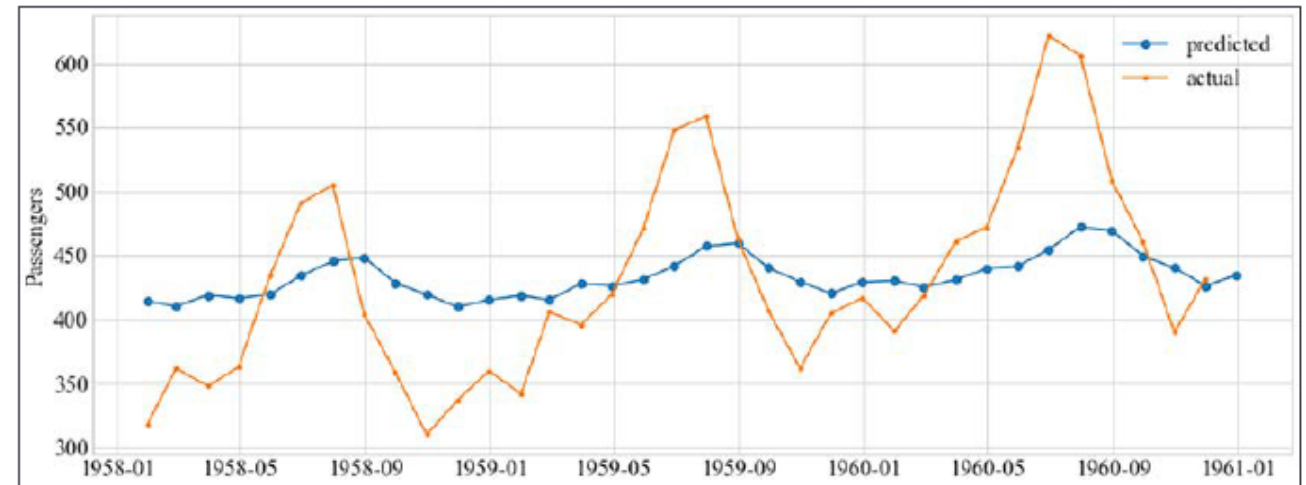


Figure 10.14: Predicted against actual airplane passengers; Fully connected network with the year and month

Improved Model

Let's create a less naïve version. We will change a few things in this model:

- We'll add an embedding of the month feature
- We'll treat the year as a linear predictor
- We'll add the previous month's passengers to our predictions
- Finally, we'll scale our predictions based on the standard deviation in the training dataset

Embedding Layer

Let's create a less naïve version. We will change a few things in this model:

- We'll add an embedding of the month feature
- We'll treat the year as a linear predictor
- We'll add the previous month's passengers to our predictions
- Finally, we'll scale our predictions based on the standard deviation in the training dataset

$$z = \frac{x - \mu}{\sigma},$$

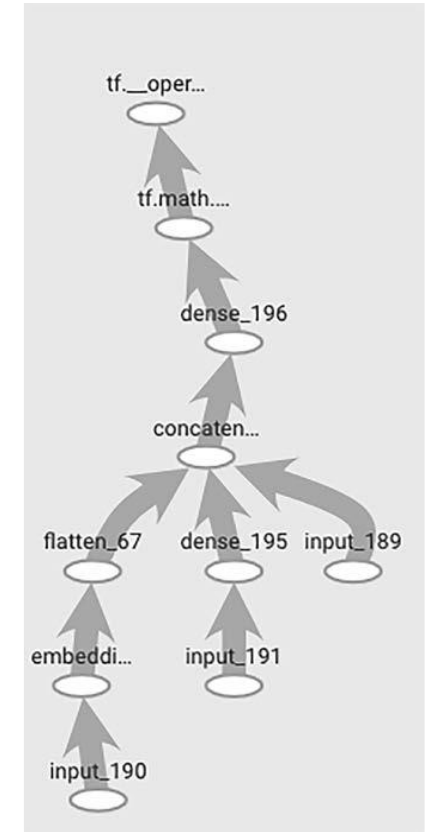
where μ is the population mean and σ is the population standard deviation.

The inverse of this is as follows:

$$x = z\sigma + \mu$$

Our formula is as follows:

$$\hat{v}_t = p\sigma + v_{t-1},$$



Implementation

```
from tensorflow.keras.layers.experimental.preprocessing
from tensorflow.keras.layers import Embedding, Flatten, Concatenate
from tensorflow.keras.metrics import (
    RootMeanSquaredError, MeanAbsoluteError
)
```

```
def create_model(train):
    scale = tf.constant(train.passengers.std())
    cont_layer = Input(shape=1)
    cat_layer = Input(shape=1)
    embedded = Embedding(12, 5)(cat_layer)
    emb_flat = Flatten()(embedded)
    year_input = Input(shape=1)
    year_layer = Dense(1)(year_input)
    hidden_output = Concatenate(-1)([emb_flat, year_layer, cont_layer])
    output_layer = keras.layers.Dense(1)(hidden_output)
    output = output_layer * scale + cont_layer
    model = keras.models.Model(inputs=[
        cont_layer, cat_layer, year_input
    ], outputs=output)
    model.compile(loss='mse', optimizer=keras.optimizers.Adam(),
        metrics=[RootMeanSquaredError(), MeanAbsoluteError()])
    return model
```

```
model = create_model(X_train)
```

```
model.fit(
    (X_train["passengers"], X_train["year"], X_train["month"]),
    y_train, epochs=1000,
    callbacks=[callback]
)
predicted = model.predict((X_test["passengers"], X_test["year"], X_
test["month"]))
```

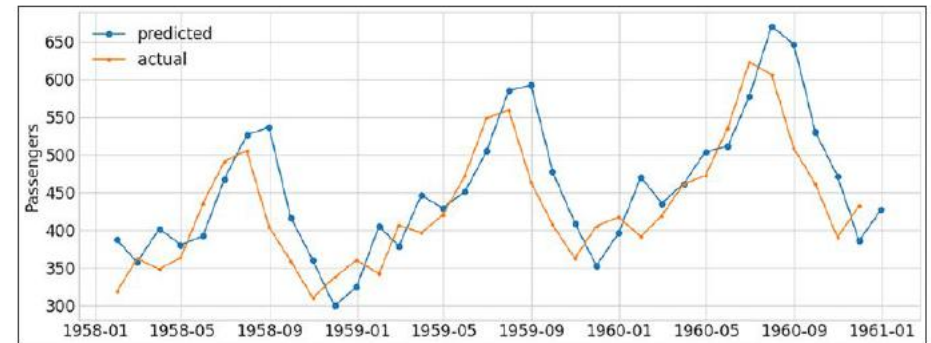


Figure 10.16: Predicted against actual airplane passenger numbers: Fully connected network with embedding, scaling, and baseline

Recurrent neural network

These **outputs** and the **hidden** state will have **encoded** the information contained in the sequence and can be used for further processing, such as to **predict** the **next step** using a **decoder**.

The **RNN** block can also be used as a **decoder** that takes in the **encoded representation** and produces the outputs.

Because of this flexibility, the RNN blocks can be arranged to suit a wide variety of input and output combinations, such as the following:

- Many-to-one, where we have many inputs and a single output—for instance, single-step forecasting or time series classification
- Many-to-many, where we have many inputs and many outputs—for instance, multi-step forecasting

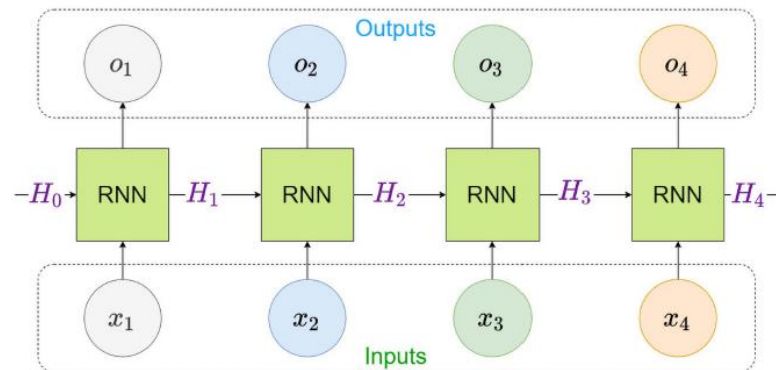


Figure 12.4: How an RNN processes input sequences

RNN in PyTorch

input_size: The number of expected features in the input. If we use just the history of the time series, this is 1. However, when we use history along with some other features, this is >1 .

- **hidden_size:** The dimension of the hidden state. This defines the size of the input-to-hidden and hidden-to-hidden matrices.
- **num_layers:** This is the number of RNNs that will be stacked on top of each other. The default is 1.
- **nonlinearity:** The non-linearity to use. Although tanh is the originally proposed non-linearity, PyTorch also allows us to use ReLU (relu). The default is tanh.
- **bias:** This parameter decides whether or not to add bias to the update equations we discussed earlier. If the parameter is False, there will be no bias. The default is True.
- **batch_first:** There are two input data configurations that the RNN cell can use—we can have the input as (*batch size, sequence length, number of features*) or (*sequence length, batch size, number of features*). `batch_first = True` selects the former as the expected input dimensions. The default is False.
- **dropout:** This parameter, if non-zero, uses a dropout layer on the outputs of each RNN layer except the last. Dropout is a popular regularization technique where randomly selected neurons are ignored during training (the *Further reading* section contains a link to the paper that proposed this). The dropout probability will be equal to dropout. The default is 0.
- **bidirectional:** This parameter enables a bidirectional RNN. If True, a bidirectional RNN is used. The default is False.

RNN in PyTorch

```
rnn = nn.RNN(  
    input_size=1,  
    hidden_size=32,  
    num_layers=1,  
    batch_first=True,  
    dropout=0,  
    bidirectional=False,  
)
```

```
#input dim: torch.Size([6, 15, 1])  
# batch size = 6, sequence length = 15 and number of features = 1, batch_first  
= True  
output, hidden_states = rnn(rnn_input)  
# output.shape -> torch.Size([6, 15, 32])  
# hidden_states.shape -> torch.Size([1, 6, 32]))
```

```
torch.equal(hidden_states[0], output[:, -1]) # -> True
```

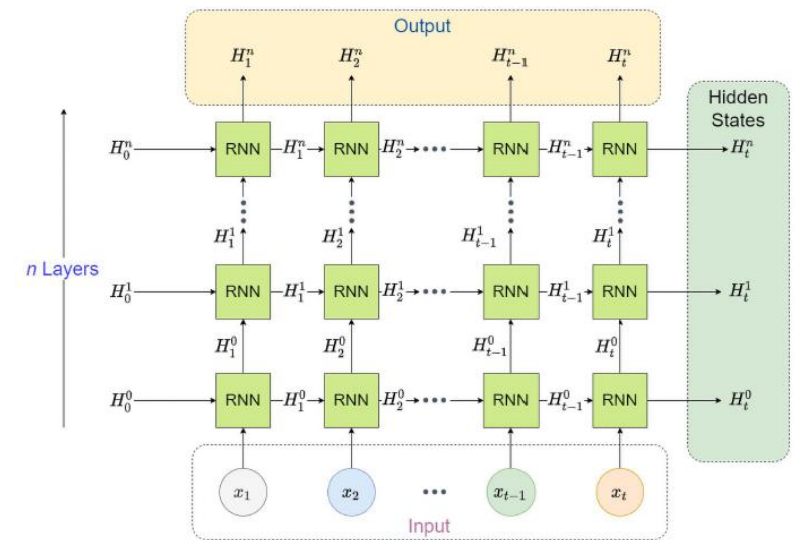
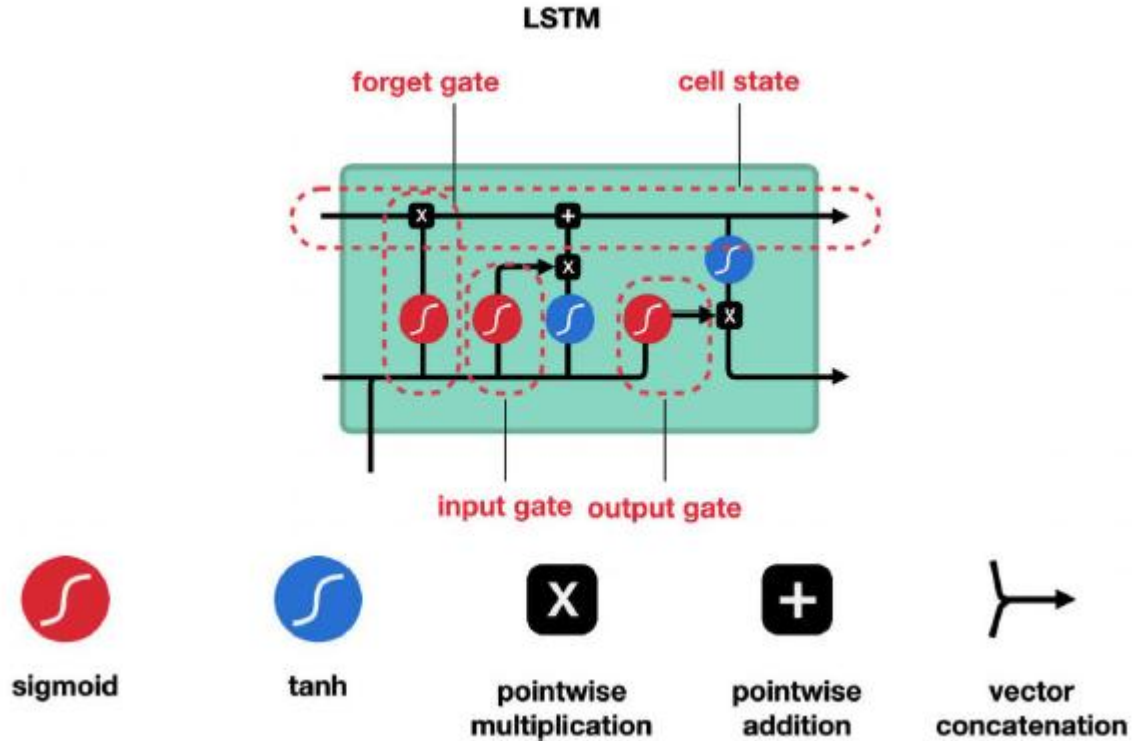


Figure 12.5: PyTorch implementation of stacked RNNs

Long short-term memory (LSTM) networks



```
lstm = nn.LSTM(  
    input_size=1,  
    hidden_size=32,  
    num_layers=5,  
    batch_first=True,  
    dropout=0,  
    # bidirectional=True,  
)  
output, (hidden_states, cell_states) = lstm(rnn_input)  
output.shape # -> [6, 15, 32]  
hidden_states.shape # -> [5, 6, 32]  
cell_states.shape # -> [5, 6, 32]
```

Recurrent neural network

```
def wrap_data(df, lookback: int):
    dataset = []
    for index in range(lookback, len(df)+1):

        features = {
            f"col_{i}": float(val) for i, val in enumerate(
                df.iloc[index-lookback:index].values
            )
        }
        row = pd.DataFrame.from_dict([features])
        row.index = [df.index[index-1]]
        dataset.append(row)
    return pd.concat(dataset, axis=0)
```

```
LOOKBACK = 10
dataset = wrap_data(passengers, lookback=LOOKBACK)
dataset = dataset.join(passengers.shift(-1))
```

```
def create_model(passengers):
    input_layer = Input(shape=(LOOKBACK, 1))
    recurrent = Bidirectional(LSTM(20, activation="tanh"))(input_layer)
    output_layer = Dense(1)(recurrent)
    model = keras.models.Model(inputs=input_layer, outputs=output_layer)
    model.compile(loss='mse', optimizer=keras.optimizers.Adagrad(),
        metrics=[keras.metrics.RootMeanSquaredError(), keras.metrics.
MeanAbsoluteError()])
    return model
```

Recurrent neural network

```
from sklearn.model_selection import train_test_split

callback = tf.keras.callbacks.EarlyStopping(monitor='loss', patience=3)
model = create_model(passengers)
X_train, X_test, y_train, y_test = train_test_split(
    dataset.drop(columns="passengers"),
    dataset["passengers"],
    shuffle=False
)

model.fit(X_train, y_train, epochs=1000, callbacks=[callback])
```

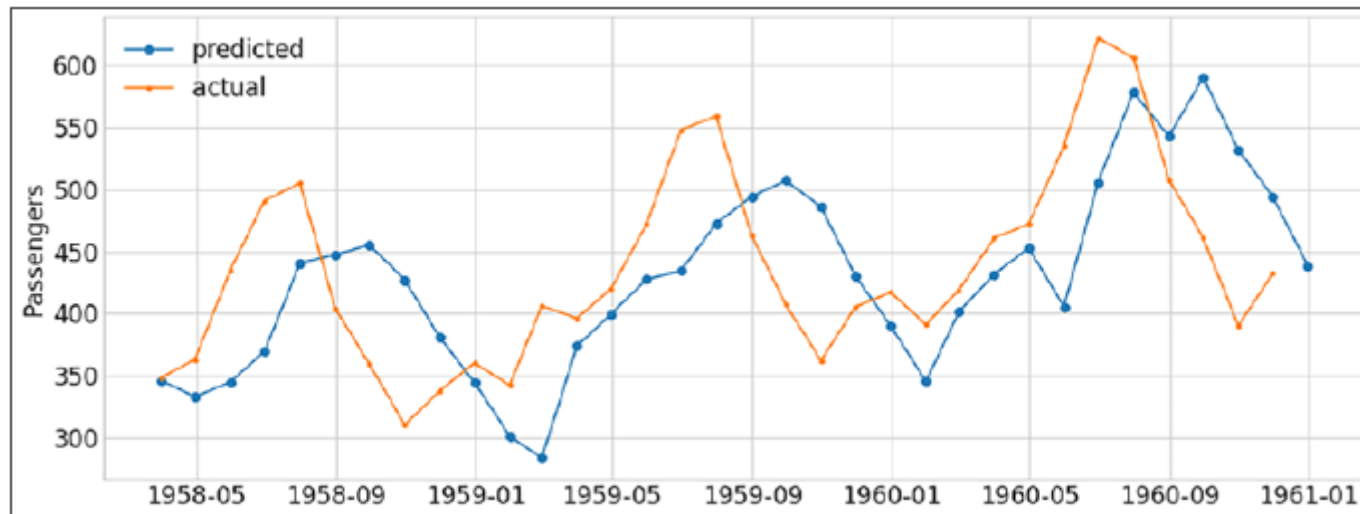
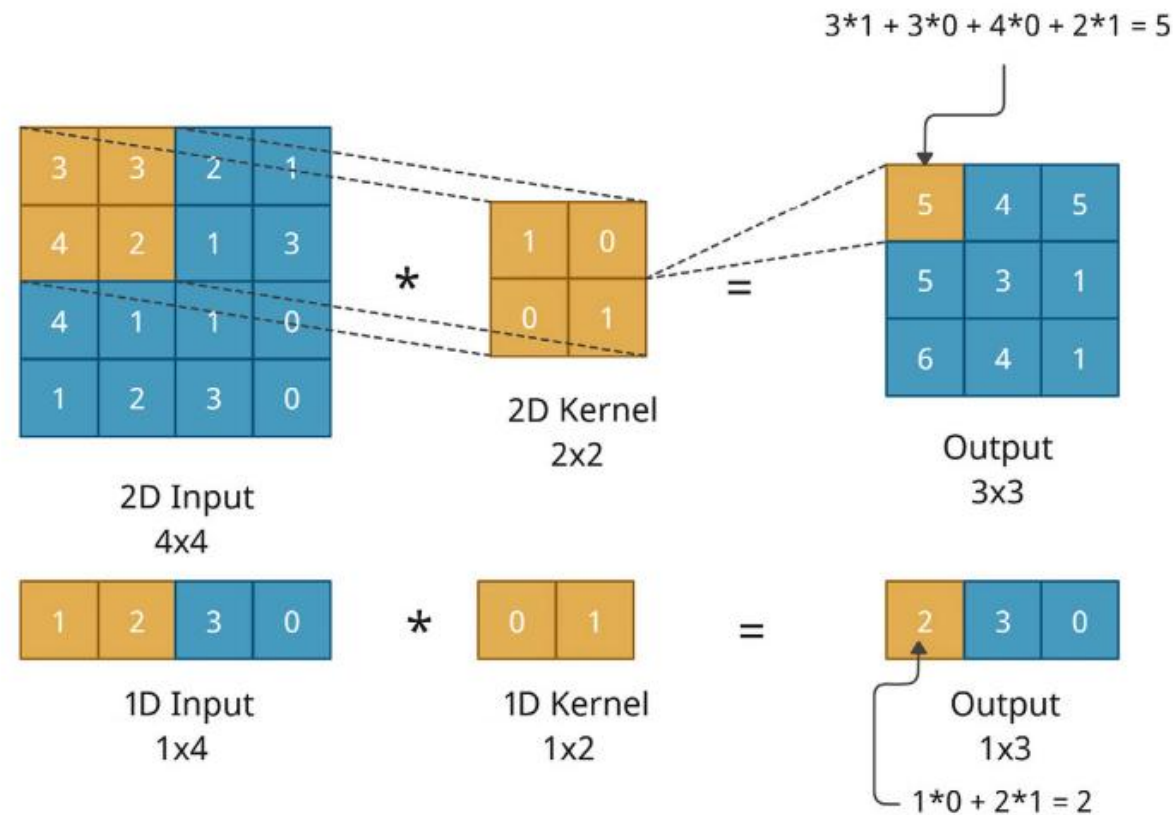


Figure 10.17: Recurrent neural network for passenger forecasts

Convolution networks

Convolution networks, also called convolutional neural networks (**CNNs**), are like neural networks for **processing data** in the form of a **grid**. This grid can be 2D (such as an image), **1D** (such as a **time series**), 3D (such as data from LIDAR sensors), and so on.

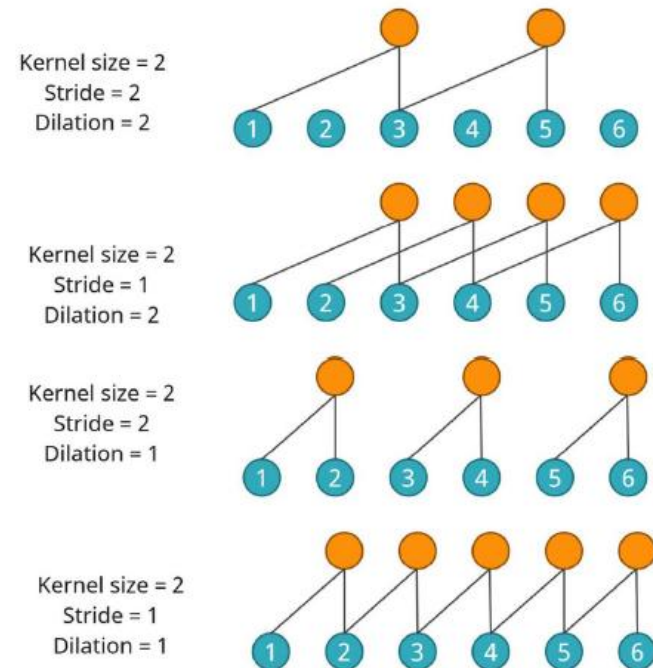


Padding, stride, and dilations

Before we start discussing these terms, let's look at an equation that gives the output dimensions (O) of a convolutional layer, given the input dimensions (L), kernel size (k), padding size (p_l for left padding and p_r for right padding), stride (s), and dilation (d):

$$O = \frac{L + p_l + p_r - d \times (k - 1) - 1}{s} + 1$$

- **Stride:** how much to slide the kernel over the sequence
- **Dilation:** how many elements in between will be skipped.



Convolution in PyTorch

- **in_channels**: Number of input features. For a single time series, it is usually 1; with extra features, it is greater than 1. In later layers, it equals the previous layer's out_channels.
- **out_channels**: Number of filters/kernels. Each filter learns different patterns from the input.
- **kernel_size**: Size of the sliding window used for convolution.
- **stride**: Step size of the convolution. Default is 1.
- **padding**: Adds values to the edges of the sequence. valid means no padding, while same keeps the output length equal to the input length.
- **padding_mode**: Defines how padding values are filled. Common options are zeros and replicate; others include reflect and circular.
- **dilation**: Spacing between kernel elements. Useful for capturing longer-range patterns. Default is 1.
- **groups**: Controls connections between input and output channels. Usually left as 1.
- **bias**: Adds a learnable bias term to the convolution. Default is True.

Convolution in PyTorch

```
conv = nn.Conv1d(in_channels=1, out_channels=1, kernel_size=k)
```

```
#input dim: torch.Size([6, 1, 15])  
# batch size = 6, number of features = 1 and sequence length = 15  
output = conv(cnn_input)  
# Output should be in_dim - k + 1  
assert output.size(-1)==cnn_input.size(-1)-k+1  
output.shape #-> torch.Size([6, 1, 13])
```

Dilated causal convolutional neural network

```
import numpy as np
import pandas as pd
from keras.layers import Conv1D, Input, Add, Activation, Dropout
from keras.models import Sequential, Model
from keras.layers.advanced_activations import LeakyReLU, ELU
from keras import optimizers
import tensorflow as tf

def DC_CNN_Block(nb_filter, filter_length, dilation):
    def f(input_):
        residual = input_
        layer_out = Conv1D(
            filters=nb_filter, kernel_size=filter_length,
            dilation_rate=dilation,
            activation='linear', padding='causal', use_bias=False
        )(input_)
        layer_out = Activation('selu')(layer_out)
        skip_out = Conv1D(1, 1, activation='linear', use_bias=False)
        (layer_out)
        network_in = Conv1D(1, 1, activation='linear', use_bias=False)
        (layer_out)

        network_out = Add()( [residual, network_in] )
        return network_out, skip_out
    return f
```


Dilated causal convolutional neural network

```
def DC_CNN_Model(length):
    input = Input(shape=(length,1))
    l1a, l1b = DC_CNN_Block(32, 2, 1)(input)
    l2a, l2b = DC_CNN_Block(32, 2, 2)(l1a)
    l3a, l3b = DC_CNN_Block(32, 2, 4)(l2a)
    l4a, l4b = DC_CNN_Block(32, 2, 8)(l3a)
    l5a, l5b = DC_CNN_Block(32, 2, 16)(l4a)
    l6a, l6b = DC_CNN_Block(32, 2, 32)(l5a)
    l6b = Dropout(0.8)(l6b)
    l7a, l7b = DC_CNN_Block(32, 2, 64)(l6a)
    l7b = Dropout(0.8)(l7b)
    l8 = Add()([l1b, l2b, l3b, l4b, l5b, l6b, l7b])
    l9 = Activation('relu')(l8)
    l21 = Conv1D(1, 1, activation='linear', use_bias=False)(l9)
    model = Model(inputs=input, outputs=l21)
    model.compile(loss='mae', optimizer=optimizers.Adam(),
metrics=['mse'])
    return model
```

```
passengers = pd.read_csv(
    "passengers.csv", parse_dates=["date "]
).set_index("date")
```

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    passengers.passengers, passengers.passengers.shift(-1),
    shuffle=False
)
```

```
def fit_model(timeseries):
    length = len(timeseries)-1
    model = DC_CNN_Model(length)
    model.summary()
    X = timeseries[:-1].reshape(1,length, 1)
    y = timeseries[1:].reshape(1,length, 1)
    model.fit(X, y, epochs=3000, callbacks=[callback])
    return model
```

Dilated causal convolutional neural network

```
def forecast(model, timeseries, horizon: int):
    length = len(timeseries)-1
    pred_array = np.zeros(horizon).reshape(1, horizon, 1)
    X_test_initial = timeseries[1:].reshape(1,length,1)
    pred_array[:,0, :] = model.predict(X_test_initial)[:, -1:, :]
    for i in range(horizon-1):
        pred_array[:, i+1:, :] = model.predict(
            np.append(
                X_test_initial[:, i+1:, :],
                pred_array[:, :i+1, :]
            ).reshape(1, length, 1))[:, -1:, :]
    return pred_array.flatten()
```

```
def evaluate_timeseries(series, horizon: int):
    model = fit_model(series)
    pred_array = forecast(model, series, horizon)
    return pred_array, model
```

```
HORIZON = len(y_test)
predictions, model = evaluate_timeseries(
    X_train.values.reshape(-1, 1), horizon= HORIZON
)
```

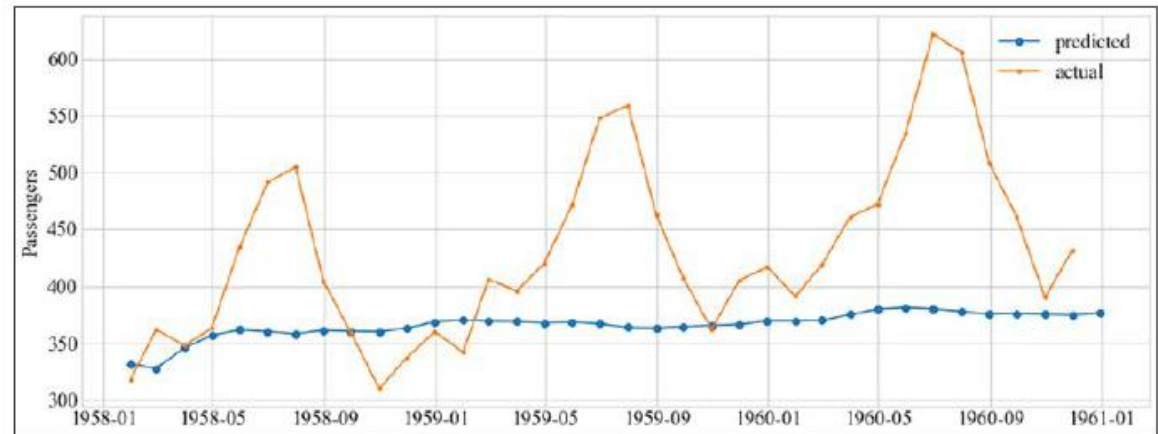
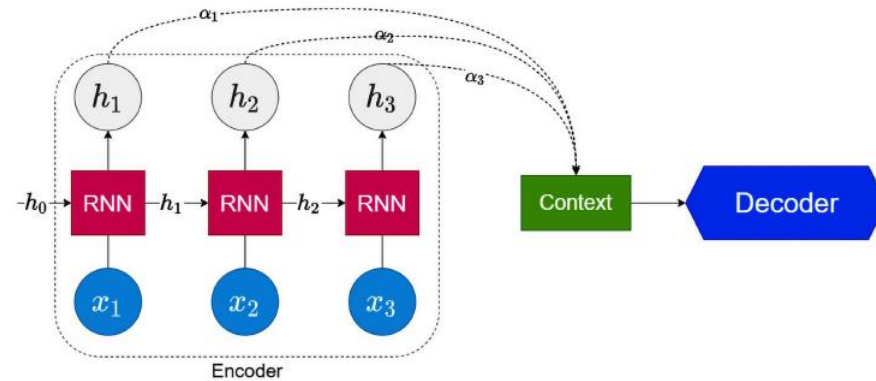
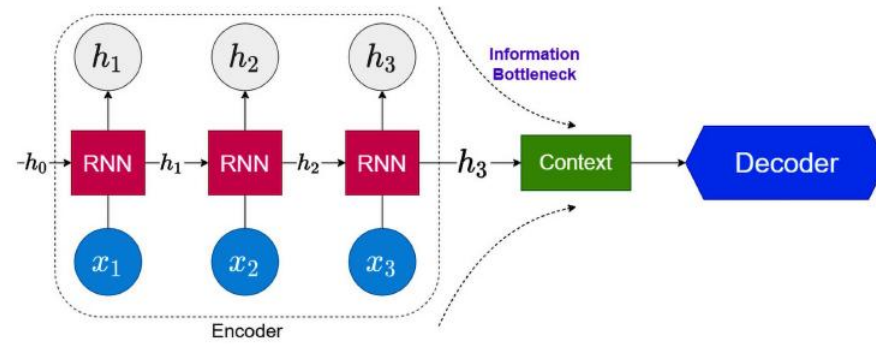
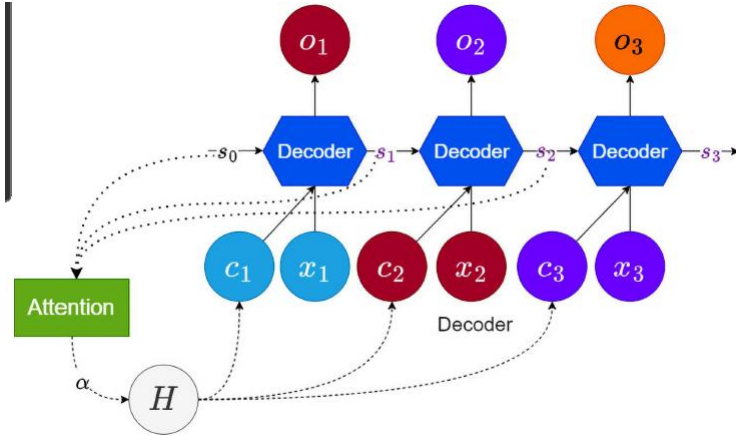


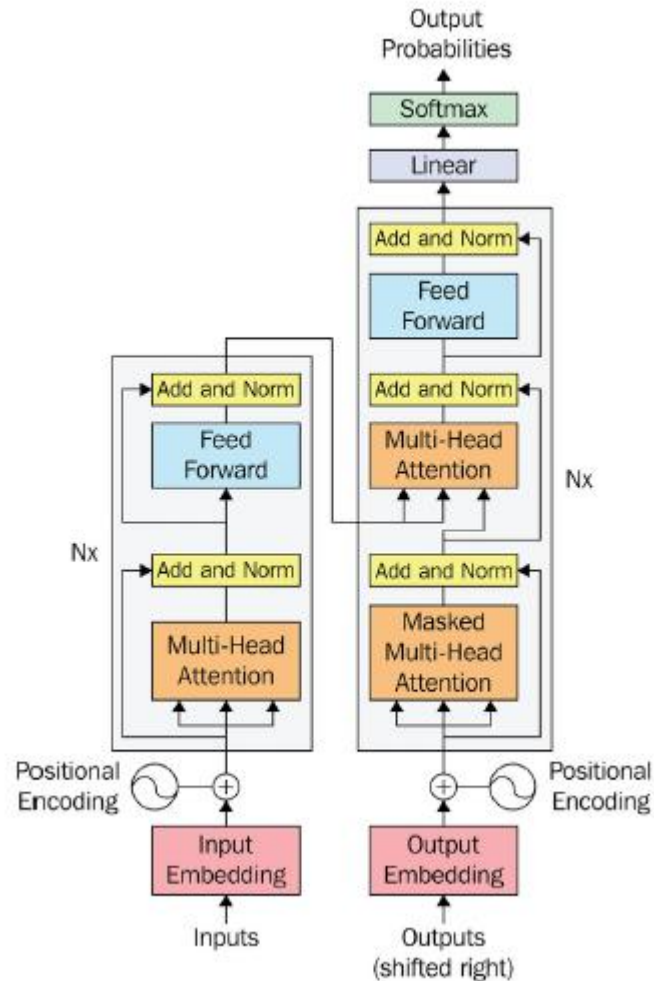
Figure 10.18: ConvNet prediction of passenger numbers

Attention



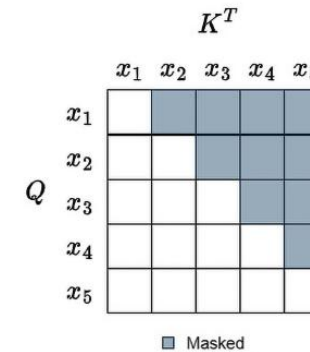
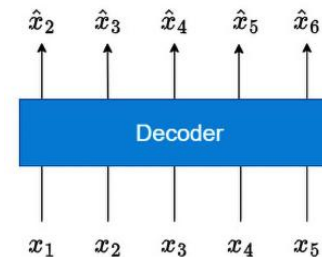
Forecasting with Transformers

Figure 1



$$p_{pos,2i} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

$$p_{pos,2i+1} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$



Forecasting with Transformers

```
# Projecting input dimension to d_model  
x_ = self.input_projection(x)  
# Adding positional encoding  
x_ = self.pos_encoder(x_)  
# Encoding the input  
x_ = self.transformer_encoder(x_, mask)  
# Decoding the input  
y_hat = self.decoder(x_)
```

```
y_hat = y_hat[:, -1, :].unsqueeze(1)
```

```
y = torch.cat([x[:, 1:, :], y], dim=1).squeeze(-1).unfold(1, y.size(1), 1)
```

Algorithm	MAE	MSE	MASE	Forecast Bias
MultiStep LSTM_LSTM_teacher_forcing_1	0.1754	0.0912	1.3671	12.93%
MultiStep_Seq2Seq_dot_Attn_teacher_forcing_1	0.1536	0.0717	1.1967	8.25%
MultiStep_Transformer_Multi_Step_FF_decoder	0.1949	0.1104	1.5188	16.24%

Forecasting with Transformers

```
self.input_projection = nn.Linear(
    self.hparams.input_size, self.hparams.d_model, bias=False
)
```

```
self.pos_encoder = PositionalEncoding(self.hparams.d_model)
```

```
    dim_feedforward=self.hparams.d_model * self.hparams.ff_multiplier,
    activation=self.hparams.activation,
    batch_first=True,
)
self.transformer_encoder = nn.TransformerEncoder(
    self.encoder_layer, num_layers=self.hparams.n_layers
)
```

```
self.decoder = nn.Sequential(nn.Linear(self.hparams.d_model, 100),
    nn.ReLU(),
    nn.Linear(100, self.hparams.multi_step_horizon)
)
```

```
mask = self._generate_square_subsequent_mask(x.shape[1]).to(x.device)
```

```
mask = (torch.triu(torch.ones(5, 5)) == 1).transpose(0, 1)
```

```
self.encoder_layer = nn.TransformerEncoderLayer(
    d_model=self.hparams.d_model,
    nhead=self.hparams.n_heads,
    dropout=self.hparams.dropout,
```

```
tensor([[ True, False, False, False, False],
        [ True,  True, False, False, False],
        [ True,  True,  True, False, False],
        [ True,  True,  True,  True, False],
        [ True,  True,  True,  True,  True]])
```

```
mask = (
    mask.float()
    .masked_fill(mask == 0, float("-inf"))
    .masked_fill(mask == 1, float(0.0))
)
```

Forecasting with Transformers

```
# Projecting input dimension to d_model  
x_ = self.input_projection(x)  
# Adding positional encoding  
x_ = self.pos_encoder(x_)  
# Encoding the input  
x_ = self.transformer_encoder(x_, mask)  
# Decoding the input  
y_hat = self.decoder(x_)
```

```
y_hat = y_hat[:, -1, :].unsqueeze(1)
```

```
y = torch.cat([x[:, 1:, :], y], dim=1).squeeze(-1).unfold(1, y.size(1), 1)
```

Algorithm	MAE	MSE	MASE	Forecast Bias
MultiStep LSTM_LSTM_teacher_forcing_1	0.1754	0.0912	1.3671	12.93%
MultiStep_Seq2Seq_dot_Attn_teacher_forcing_1	0.1536	0.0717	1.1967	8.25%
MultiStep_Transformer_Multi_Step_FF_decoder	0.1949	0.1104	1.5188	16.24%